ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

ΣΧΟΛΗ ΤΕΧΝΟΛΟΓΙΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΕΠΙΚΟΙΝΩΝΙΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

Πτυχιακή Εργασία

Τίτλος Πτυχιακής Εργασίας	Παιχνίδι σκακιού με online multiplayer για Android και Windows Online multiplayer chess game for Android and Windows
Ονοματεπώνυμο Φοιτητή	ΑΠΌΣΤΟΛΟΣ ΣΊΨΑΣ
Πατρώνυμο	ΣΏΤΉΡΗΣ
Αριθμός Μητρώου	Π/ …19148……..
Επιβλέπων	Αλέπης Ευθύμιος

Ημερομηνία Παράδοσης 3/2026

Περίληψη

Η εφαρμογή Chess Online είναι ένα παιχνίδι σκακιού δύο παικτών, αναπτυγμένο σε Unity με γλώσσα C#. Υποστηρίζει

- τοπικό παιχνίδι (2 παίκτες στην ίδια συσκευή) και online
- παιχνίδι μέσω LAN Wi-Fi χωρίς ανάγκη σύνδεσης στο διαδίκτυο.

Το σύστημα networking βασίζεται στο Unity Transport (UDP),ενώ η ανακάλυψη lobby γίνεται μέσω διπλού UDP μηχανισμού (παθητικό broadcast + ενεργό probe/response) που εξασφαλίζει λειτουργία τόσο σε Android όσο και σε Windows παρά τους

περιορισμούς του Windows Firewall.

Η σκακιστική λογική υλοποιείται πλήρως με OOP αρχιτεκτονική (βασική κλάση Piece + υποκλάσεις ανά κομμάτι) και καλύπτει όλους τους επίσημους κανόνες: κινήσεις κομματιών:

- σαχ
- ματ
- ισοπαλία
- Ροκέ
- En Passant
- Προαγωγή Πιονιού.

Πρόσθετες λειτουργίες: σύστημα πρότασης ισοπαλίας με χρονικό cooldown (15 δευτερόλεπτα) και όριο (3 προσφορές ανά παρτίδα), προστασία lobby με 4ψήφιο PIN, αυτόματο σύστημα rematch, και ειδοποίηση αποχώρησης αντιπάλου.

Λέξεις-κλειδιά: Unity, C#, Σκάκι, Networking, LAN, UDP, Android, Windows, Client-Server, Real-Time Multiplayer.

Πίνακας Περιεχομένων

Περίληψη

1. Προβλήματα και Προσεγγίσεις

1.1 Επιλογή Unity ως Game Engine

1.2 Σχεδίαση Multiplayer Real-Time Server-Client

1.3 Σχεδίαση Cross-Platform (Android + Windows)

2. Προδιαγραφές και Ανάλυση Απαιτήσεω

2.1 Υλικές απαιτήσεις

2.2 Απαιτήσεις λογισμικού

2.3 Ανάλυση θεματολογίας εφαρμογής

2.4 Προκαταρκτική ανάλυση εφαρμογής

2.5 Ανάλυση κανόνων σκακιού

2.6 Ανάλυση χρήστη – Guest Mode

2.7 Λειτουργικές και μη λειτουργικές απαιτήσεις .

3. Σχεδίαση Εφαρμογής

3.1 UI / Menu System

3.1.1 Start Menu

3.1.2 Host Menu

3.1.3 Lobby Menu (JOIN)

3.1.4 In-Game *Options*

3.1.5 Winner Canvas

3.2 Chess Board & Piece System

3.2.1 Board Initialization

3.2.2 Piece Classes και Subclasses

3.2.3 Movement Rules και Validation

3.3 Networking System

3.3.1 NetMessages και υποκλάσεις

3.3.2 LobbyManager

3.3.3 Client-Server Flow

3.3.4 Synchronization & State Management

4. Υλοποίηση Εφαρμογής

4.1 Παρουσίαση scripts UI

4.2 Παρουσίαση scripts Chess Logic

4.3 Παρουσίαση scripts Networking

4.4 GameManager και κεντρικός έλεγχος

4.5 Cross-platform build & testing

5. Εκτέλεση

5.1 Αρχική οθόνη
5.2 Τοπικό Παιχνίδι
5.3 Παιχνίδι Μέσω Δικτύου (Online Game)
5.3.1 Μενού Δημιουργίας Δωματίου (Host Menu)
5.3.2 Connect Menu

5.4 Online In-Game

5.5 Ειδικές κινήσεις

6. Συμπεράσματα

6.1 Σύγκριση στόχων με τελικό αποτέλεσμα
6.2 Απόκτηση γνώσεων

6.3 Περιορισμοί εφαρμογής
6.4 Προτάσεις μελλοντικής εξέλιξης

7. Βιβλιογραφία & Παραρτήματα

7.1 Βιβλιογραφία

7.2 Video

7.3 Scripts

8. Περαιτέρω σύνδεσμοι και αρχεία
9. Ευχαριστίες

1. Προβλήματα και Προσεγγίσεις

1.1 Επιλογή Unity ως Game Engine

Η Unity επιλέχθηκε ως η πλατφόρμα ανάπτυξης του έργου για

τους εξής λόγους:

- Cross-platform build: Μέσα από ένα μόνο codebase παράγονται εκτελέσιμα για Android (.apk) και Windows (.exe) χωρίς επιπλέον ανάπτυξη.
- Ενσωματωμένο σύστημα 3D renderer: Το ταμπλό σκακιού και τα κομμάτια αποδίδονται σε 3D χώρο με Materials, Camera και Lighting που διαχειρίζεται η ίδια η μηχανή.
- Unity Networking Transport: Παρέχει χαμηλού επιπέδου UDP δικτύωση (NetworkDriver) που ελέγχεται πλήρως από τον προγραμματιστή, χωρίς εξάρτηση από εξωτερικό relay server.
- C# ως γλώσσα: Ισχυρή με πλούσια βιβλιοθήκη (.NET / Mono) που επιτρέπει τόσο game logic (MonoBehaviour) όσο και low-level networking (System.Net).
- Δωρεάν για indie/ακαδημαϊκή χρήση: Δεν απαιτεί άδεια για έργα μη εμπορικής φύσης.

Εναλλακτικές που εξετάστηκαν και απορρίφθηκαν:

- Godot: Μικρότερο ecosystem, λιγότερη τεκμηρίωση για mobile networking.
- Pygame (Python): Δεν υποστηρίζει native Android builds.
- Web (JavaScript/HTML5): Περιορισμοί στη χρήση raw UDP sockets.

1.2 Σχεδίαση Multiplayer Real-Time Server-Client

Η αρχιτεκτονική επιλέχθηκε να είναι peer-to-peer με έναν από τους δύο παίκτες να λειτουργεί ως server (HOST) και τον άλλον ως client (JOIN). Αυτό αποφεύγει την ανάγκη για dedicated server υποδομή.

Προβλήματα που αντιμετωπίστηκαν:

1. **Windows Firewall:** Σε Windows περιβάλλον, το Firewall μπλοκάρει εισερχόμενα UDPπακέτα από Android συσκευές. Λύση: Διπλός μηχανισμός ανακάλυψης (παθητικός broadcast port 8008 + ενεργός probe/response port 8009).O client στέλνει πρώτος "CHESS_DISCOVER" — η απάντηση του host επιτρέπεται εισερχόμενη γιατί το socket εκκίνησε εξερχόμενα. Επιπλέον, η τεχνική "PrePunchGamePort" ανοίγει εκ των προτέρων τη θύρα παιχνιδιού (8007) στο Windows Firewall.
2. **Thread-safety**: Τα UDP threads εκτελούνται εκτός Unity main thread. Η UI ενημέρωση γίνεται μέσω MainThreadDispatcher (concurrent queue που εκτελεί actions στο Unity Update loop).
3. **Synchronization:** Η κατάσταση παιχνιδιού διατηρείται στο HOST και μεταδίδεται στον client μέσω NetMessages (άθροισμα κινήσεων, όχι full-state sync), κάτι που είναι αρκετό για σκάκι (turn-based, deterministic).
4. **Disconnect handling**: Χρονόμετρο αποσύνδεσης 5 δευτερόλεπτα (disconnectTimeoutMS). KeepAlive μηνύματα κάθε 20 δευτερόλεπτα διατηρούν τη σύνδεση ενεργή.

1.3 Σχεδίαση Cross-Platform (Android + Windows)

Η cross-platform ανάπτυξη παρουσίασε τις εξής προκλήσεις:

α) Input:

Στο PC χρησιμοποιείται mouse (Ray από Camera), στο Android touch events (GetTouch). Ο κώδικας χειρίζεται και τα δύο ενοποιημένα στον Update loop της ChessBord.

β) UI Layout: Όλη η διεπαφή χρηστών κατασκευάζεται δυναμικά σε runtime με C# (VerticalLayoutGroup, HorizontalLayoutGroup,ContentSizeFitter) αντί για Unity Prefabs, ώστε να προσαρμόζεται αυτόματα στο μέγεθος οθόνης.

γ) Δίκτυο: Το Android επιτρέπει broadcast από τρίτες εφαρμογές πιο ελεύθερα από τα Windows. Ο κώδικας χρησιμοποιεί fallback-πολλαπλές στρατηγικές για IP ανακάλυψη (UDP trick, NetworkInterface enumeration) για αξιόπιστη λειτουργία και στις δύο πλατφόρμες

δ**) Build Settings:** Το project χρειάζεται ξεχωριστές ρυθμίσεις

στο Unity Build Settings για Android (ARM64, IL2CPP) και

Windows (x64, Mono ή IL2CPP).

2. Προδιαγραφές και Ανάλυση Απαιτήσεων

2.1 Υλικές απαιτήσεις

Ελάχιστες απαιτήσεις Android:

- Android 8.0 (API Level 26) ή νεότερο
- RAM: 2 GB

- Αποθηκευτικός χώρος: 150 MB
- Wi-Fi προσαρμογέας (για online παιχνίδι)
- Επεξεργαστής: Quad-core 1.8 GHz

Ελάχιστες απαιτήσεις Windows:

- Windows 10 ή Windows 11 (64-bit)
- RAM: 4 GB
- Κάρτα γραφικών: DirectX 11 συμβατή
- Αποθηκευτικός χώρος: 200 MB
- Wi-Fi / Ethernet (για online παιχνίδι)

Δίκτυο:

- Και οι δύο συσκευές πρέπει να βρίσκονται στο ίδιο τοπικό δίκτυο (Wi-Fi router ή hotspot)
- Δεν απαιτείται σύνδεση στο διαδίκτυο
- Θύρες δικτύου: 8007, 8008, 8009, 8010 (UDP)

2.2 Απαιτήσεις λογισμικού

Για ανάπτυξη:

- Unity 2022.x ή νεότερο (LTS προτεινόμενο)
- Unity Package: com.unity.transport (1.x ή 2.x)
- Unity Package: com.unity.textmeshpro
- Visual Studio 2022 ή JetBrains Rider
- Android SDK (για Android builds)
- JDK 11+ (για Android builds)

<u>Για εκτέλεση:</u>

- Android: Κανένα πρόσθετο λογισμικό
- Windows: .NET Runtime ενσωματωμένο στο build
- Windows (προαιρετικό): Εκτέλεση ως Διαχειριστής αν το Windows Firewall μπλοκάρει τις θύρες UDP

2.3 Ανάλυση θεματολογίας εφαρμογής

Το σκάκι (Chess) είναι ένα από τα παλαιότερα και πιο διάσημα επιτραπέζια παιχνίδια στρατηγικής στον κόσμο, με ιστορία άνω των 1.500 ετών. Παίζεται από 2 παίκτες σε ταμπλό 8×8 τετραγώνων με 32 κομμάτια (16 ανά πλευρά). Στόχος: Να θέσεις τον εχθρικό Βασιλιά σε θέση από την οποία δεν μπορεί να διαφύγει (Ματ / Checkmate).

Η ψηφιακή μορφή του σκακιού είναι δημοφιλής σε όλο τον κόσμο, με πλατφόρμες όπως το Chess.com να συγκεντρώνουν εκατομμύρια χρήστες. Η παρούσα εφαρμογή εστιάζει στο τοπικό multiplayer, κατάλληλο για φίλους .

2.4 Προκαταρκτική ανάλυση εφαρμογής

Η εφαρμογή αποτελείται από τρία βασικά υποσυστήματα:

1. Σκακιστική Λογική (Chess Logic):

Πλήρης υλοποίηση κανόνων σκακιού — κινήσεις κομματιών, έλεγχος εγκυρότητας, ανίχνευση σαχ/ματ, ειδικές κινήσεις.

2. Networking:

Server-Client αρχιτεκτονική πάνω από Unity Transport, αυτόματη ανακάλυψη host στο τοπικό δίκτυο, ανταλλαγή μηνυμάτων παιχνιδιού.

3. Διεπαφή Χρηστών (UI):

Δυναμική κατασκευή όλων των panels/buttons/menus σε runtime με C# και TextMeshPro.

2.5 Ανάλυση κανόνων σκακιού

Τα 6 είδη κομματιών και οι κινήσεις τους:

- **Πιόνι (Pawn):**

  - Κινείται 1 τετράγωνο μπροστά (ή 2 από αρχική θέση)

  - Χτυπά διαγώνια (1 τετράγωνο μπροστά-πλάγια)

  - Ειδική κίνηση: En Passant (βλ. παρακάτω)

  - Ειδική κίνηση: Προαγωγή (βλ. παρακάτω)

- **Ίππος (Knight):**

  - Κινείται σε σχήμα L: 2 τετράγωνα + 1 κάθετα

  - Μοναδικό κομμάτι που "πηδά" πάνω από άλλα

- **Αξιωματικός (Bishop):**

  - Κινείται απεριόριστα διαγώνια

- Παραμένει πάντα σε τετράγωνα ίδιου χρώματος

- **Πύργος (Rook):**

  - Κινείται απεριόριστα οριζόντια ή κάθετα

  - Ειδική κίνηση: Ροκέ (βλ. παρακάτω)

- **Βασίλισσα (Queen):**

  - Συνδυάζει Πύργο + Αξιωματικό

  - Πιο ισχυρό κομμάτι

- **Βασιλιάς (King):**

  - Κινείται 1 τετράγωνο προς οποιαδήποτε κατεύθυνση

  - Ειδική κίνηση: Ροκέ (βλ. παρακάτω)

- **Ειδικές κινήσεις:**

  - Ροκέ (Castling):

    Ο Βασιλιάς κινείται 2 τετράγωνα προς Πύργο, ο Πύργος τοποθετείται στην αντίπερα πλευρά. Απαιτεί:

    - Κανένα από τα δύο να μην έχει κινηθεί προηγουμένως

    - Κενά τετράγωνα μεταξύ τους

    - Ο Βασιλιάς να μην βρίσκεται σε σαχ

    - Ο Βασιλιάς να μην διασχίζει τετράγωνο υπό επίθεση

  - En Passant:

    Αν εχθρικό πιόνι κινηθεί 2 τετράγωνα (πρώτη κίνηση) και σταθεί δίπλα σε δικό σου πιόνι, μπορείς να το χτυπήσεις διαγώνια σαν να είχε κινηθεί μόνο 1 τετράγωνο.Ισχύει ΑΠΟΚΛΕΙΣΤΙΚΑ στην αμέσως επόμενη κίνηση.

  - Προαγωγή Πιονιού (Promotion):

Όταν πιόνι φτάσει στην τελευταία γραμμή του αντιπάλου, μετατρέπεται αυτόματα σε Βασίλισσα (υλοποίηση αυτόματης προαγωγής χωρίς επιλογή).

- Σαχ (Check) και Ματ (Checkmate):

` Η εφαρμογή ανιχνεύει αν ο Βασιλιάς τίθεται υπό επίθεση (σαχ) και αποκλείει κινήσεις που αφήνουν ή βάζουν το ίδιο Βασιλιά σε σαχ. Αν δεν υπάρχει καμία έγκυρη κίνηση, κηρύσσεται ματ.

2.6 Ανάλυση χρήστη – Guest Mode

Η εφαρμογή λειτουργεί αποκλειστικά σε Guest Mode. Δεν απαιτείται εγγραφή ή σύνδεση σε λογαριασμό.

Προφίλ χρήστη:

- Ηλικία: 10+ έτη (οποιοσδήποτε γνωρίζει σκάκι)

- Τεχνικές γνώσεις: Βασικές (χρήση κινητού/PC)

- Γλωσσικό επίπεδο: Η διεπαφή είναι αγγλική, η τεκμηρίωση ελληνική

- Σενάριο χρήσης: Δύο άτομα θέλουν να παίξουν σκάκι άμεσα χωρίς εγγραφή

Ο ρόλος "HOST" (επιλογή ονόματος, πιθανά και pin), ενώ ο ρόλος "JOIN" είναι πλήρως αυτοματοποιημένος (μόνο επιλογή από λίστα).

2.7 Λειτουργικές και μη λειτουργικές απαιτήσεις

Λειτουργικές απαιτήσεις (Functional Requirements):

- Τοπικό παιχνίδι 2 παικτών στην ίδια συσκευή
- Online παιχνίδι 2 παικτών μέσω LAN χωρίς internet
- Πλήρης υλοποίηση κανόνων σκακιού (βλ. §2.5)

- Αυτόματη ανακάλυψη lobby στο δίκτυο
- Δυνατότητα ονοματοδοσίας lobby
- Προστασία lobby με 4ψήφιο PIN
- Πρόταση ισοπαλίας με αποδοχή/απόρριψη
- Cooldown 15 δευτερολέπτων μετά από απόρριψη ισοπαλίας
- Μέγιστο 3 προσφορές ισοπαλίας ανά παρτίδα (DRAW LIMIT)
- Δυνατότητα rematch με εναλλαγή χρωμάτων
- Ειδοποίηση αποχώρησης αντιπάλου
- Ειδική κίνηση Ροκέ (μεγάλο και μικρό)
- Ειδική κίνηση En Passant
- Προαγωγή Πιονιού σε Βασίλισσα

Μη λειτουργικές απαιτήσεις (Non-Functional Requirements):

- Χρόνος απόκρισης κίνησης < 200ms σε τοπικό δίκτυο
- Αυτόματη ανακάλυψη lobby εντός 10 δευτερολέπτων
- Σταθερή λειτουργία με disconnectTimeout 5s
- Χρήση ελάχιστης μνήμης (χωρίς 3D resources loading)
- Ευανάγνωστος κώδικας με σχόλια στην ελληνική γλώσσα
- Λειτουργία χωρίς root/admin δικαιώματα σε Android
- Cross-platform build από ενιαίο codebase Unity

3. Σχεδίαση Εφαρμογής

3.1 UI / Menu System

Η διεπαφή χρήστη της εφαρμογής έχει σχεδιαστεί με **μικτό σύστημα** δημιουργίας UI: κάποια στοιχεία κατασκευάζονται πλήρως δυναμικά κατά τη διάρκεια εκτέλεσης (runtime) μέσω C# scripts, ενώ άλλα βασίζονται σε Unity Prefabs, ώστε να επιτυγχάνεται ένας συνδυασμός προσαρμοστικότητας και επαναχρησιμοποίησης. Το script Game UI.cs διαχειρίζεται την κατασκευή και την ενημέρωση όλων των panels,

κουμπιών, scroll views και input fields, χρησιμοποιώντας και τις δύο προσεγγίσεις.

Στα στοιχεία που δημιουργούνται runtime, χρησιμοποιούνται οι εξής τεχνικές:

- **Δημιουργία GameObject**: new GameObject() για panels ή κουμπιά, με προσθήκη components μέσω AddComponent<>().
- **Διάταξη στοιχείων**: VerticalLayoutGroup και HorizontalLayoutGroup για αυτόματη και συμμετρική τοποθέτηση UI στοιχείων.
- **Αυτόματη προσαρμογή μεγέθους**: ContentSizeFitter για να προσαρμόζονται panels και κουμπιά στις διαστάσεις της οθόνης.
- **Κείμενο υψηλής ποιότητας**: TextMeshPro (TMP_Text, TMP_InputField) για ευκρινή εμφάνιση όλων των κειμένων και οδηγών.

Στα στοιχεία που χρησιμοποιούν **Prefabs**, η εφαρμογή εκμεταλλεύεται την **προκαθορισμένη δομή και σχεδίαση**, όπως:

- Κουμπιά με προκαθορισμένα χρώματα, animations και hover effects
- Scroll view entries που εμφανίζουν ονόματα lobby και ένδειξη PIN
- Panels αποτελεσμάτων νίκης ή draw panels με πλήρως σχεδιασμένο layout

Αυτός ο συνδυασμός επιτρέπει:

1. **Μέγιστη ευελιξία**: τα στοιχεία runtime μπορούν να προσαρμόζονται δυναμικά σε διαφορετικές αναλύσεις οθόνης και συσκευές.
2. **Επαναχρησιμοποίηση**: τα Prefabs μειώνουν την ανάγκη επανασχεδιασμού για επαναλαμβανόμενα στοιχεία UI και εξασφαλίζουν οπτική συνέπεια σε όλο το παιχνίδι.

Με αυτή τη στρατηγική, η εφαρμογή μπορεί να προσαρμόζεται εύκολα σε διαφορετικά μεγέθη οθόνης, χωρίς να χάνει την **ακρίβεια και συνέπεια της διεπαφής**.

- **3.1.1 Start Menu**

Το **κύριο μενού εκκίνησης (Start Menu)** περιλαμβάνει τρία βασικά κουμπιά: LOCAL, HOST και JOIN, τοποθετημένα σε ένα **κεντρικό VerticalLayout** για συμμετρική και εύκολη πλοήγηση. Στην κορυφή εμφανίζεται το λογότυπο της εφαρμογής με τον τίτλο **"Let's Chess"**, προσδίδοντας επαγγελματική εμφάνιση και άμεση αναγνωρισιμότητα.

Κάθε φορά που εμφανίζεται το Start Menu, ο **Client** και ο **Server** απενεργοποιούνται αυτόματα, ώστε να γίνεται **καθαρισμός προηγούμενων συνδέσεων**, αποφεύγοντας την εμφάνιση σφαλμάτων ή παλαιών δεδομένων σύνδεσης και διασφαλίζοντας την ομαλή λειτουργία της εφαρμογής από την αρχή.

- **3.1.2 Host Menu**

Το menu για τη δημιουργία ενός **lobby** περιλαμβάνει ένα panel με τα εξής στοιχεία:

- InputField για το όνομα του lobby (ContentType: Standard), ώστε ο χρήστης να μπορεί να δώσει **προσαρμοσμένο όνομα**
- InputField για **PIN** (ContentType: IntegerNumber, μέγιστο 4 ψηφία), για προστασία του lobby από μη εξουσιοδοτημένους παίκτες
- Κουμπί CONFIRM για την εκκίνηση του Server και ενεργοποίηση του lobby

Με το πάτημα του κουμπιού **CONFIRM**, καλείται η μέθοδος Server.Init() με το hostname και το PIN που εισήγαγε ο χρήστης. Παράλληλα, ξεκινά το **LobbyDiscovery.StartBroadcasting()**, το οποίο ανακοινώνει την παρουσία του lobby στο τοπικό δίκτυο μέσω UDP broadcasts, επιτρέποντας στους υπόλοιπους παίκτες να εντοπίσουν εύκολα το lobby και να συνδεθούν γρήγορα.

- **3.1.3 Lobby Menu (JOIN)**

Η οθόνη σύνδεσης σε lobby χρησιμοποιεί ένα **ScrollView** που εμφανίζει δυναμικά όλα τα **ανακαλυφθέντα lobby**. Κάθε εγγραφή (entry) εμφανίζει:

- Το **όνομα του lobby**

- Ετικέτα [PIN] με χρυσό χρώμα αν το lobby προστατεύεται με κωδικό, για οπτική αναγνώριση
- Κουμπί CONNECT για άμεση σύνδεση με το επιλεγμένο lobby

Η λίστα ενημερώνεται αυτόματα μέσω του **event** LobbyDiscovery.OnHostListChanged, διασφαλίζοντας ότι εμφανίζονται μόνο **ενεργά και φρέσκα lobby**. Παράλληλα, υπάρχει κουμπί BACK για επιστροφή στο Start Menu. Το σύστημα αυτό καθιστά τη σύνδεση απλή, ακόμα και για χρήστες που δεν γνωρίζουν διευθύνσεις IP ή τεχνικές λεπτομέρειες δικτύου.

- **3.1.4 In-Game Options**

Κατά τη διάρκεια της παρτίδας, στην **κάτω μπάρα** της οθόνης εμφανίζονται:

- Κουμπί OFFER DRAW για πρόταση ισοπαλίας, με ένδειξη ενεργό/ανενεργό και περιορισμό DRAW LIMIT
- Κουμπί LEAVE GAME για άμεση αποχώρηση από το παιχνίδι

Σε περίπτωση που ο αντίπαλος προτείνει ισοπαλία, εμφανίζεται ένα **ξεχωριστό panel** με κουμπιά **ACCEPT** και **DECLINE**, ώστε ο χρήστης να μπορεί να αποδεχθεί ή να απορρίψει την πρόταση με σαφή και εύκολη επιλογή.

- **3.1.5 Winner Canvas**

Στο τέλος κάθε παρτίδας, ένα **full-screen overlay** εμφανίζει το αποτέλεσμα της παρτίδας με:

- Μήνυμα νίκης, ισοπαλίας ή αποχώρησης
- Κουμπί REMATCH, με ένδειξη αν ο αντίπαλος περιμένει
- Κουμπί MENU για επιστροφή στο Start Menu
- Ξεχωριστό drawWinPanel για την εμφάνιση αποτελέσματος ισοπαλίας

Αυτό το panel εξασφαλίζει **καθαρή και άμεση ανατροφοδότηση στον χρήστη**, παρέχοντας όλες τις απαραίτητες επιλογές για συνέχιση ή έξοδο από το παιχνίδι.

3,2

3.2.1 Board Initialization

Η σκακιέρα (8×8) αρχικοποιείται στη μέθοδο GenerateAllTiles() της ChessBord. Κάθε τετράγωνο είναι ένα GameObject με BoxCollider για ray casting. Η κάμερα στοχεύει στο κέντρο του ταμπλό (boardCenter). Τα κομμάτια τοποθετούνται με SpawnAllPieces() σε σταθερές θέσεις αρχικής διάταξης. Για online παιχνίδι, ο client βλέπει το ταμπλό αναστραμμένο (180° rotation) ώστε τα κομμάτια του να είναι πάντα στο κάτω μέρος.

3.2.2 Piece Classes και Subclasses

Βασική κλάση: Piece.cs

- team (0=Λευκό, 1=Μαύρο), currentX, currentY

- abstract GetAvailableMoves() → List<Vector2Int>

- abstract GetSpecialMoves() → SpecialMove enum

Υποκλάσεις (ανά κομμάτι):

- Pawn.cs — κίνηση +1/+2, χτύπημα διαγώνια, En Passant
- Rook.cs — 4 κατευθύνσεις (↑ ↓ ←→)
- Knight.cs — 8 L-θέσεις
- Bishop.cs — 4 διαγώνιες κατευθύνσεις
- Queen.cs — 8 κατευθύνσεις (Rook + Bishop)
- King.cs — 8 διαγώνιες/ευθείες + Castling

3.2.3 Movement Rules και Validation

Κάθε Piece.GetAvailableMoves() επιστρέφει "raw" κινήσεις.

Πριν εκτελεστεί κίνηση:

1. Φιλτράρονται κινήσεις που αφήνουν/βάζουν τον ίδιο Βασιλιά σε σαχ (ContainsValidMoves μέθοδος).

2. Ελέγχεται αν μετά την κίνηση του αντιπάλου υπάρχει Ματ (CheckForCheckmate) ή Ισοπαλία (stalemate).

3. Ειδικές κινήσεις υπολογίζονται ξεχωριστά στη ProcessSpecialMove().

3,3 Networking System

3.3.1 NetMessages και υποκλάσεις

Η βάση NetMessage ορίζει 4 virtual μεθόδους:

- Serialize(ref DataStreamWriter) → γράφει τα δεδομένα
- Deserialize(DataStreamReader) → διαβάζει τα δεδομένα
- ReceivedOnClient() → ενεργοποιεί C_* event
- ReceivedOnServer(NetworkConnection) → ενεργοποιεί S_* event

Υποκλάσεις (OpCode):

- NetKeepAlive — heartbeat για διατήρηση σύνδεσης
- NetWelcome — client → server, αρχική χειραψία
- NetStartGame — server → client, έναρξη παρτίδας
- NetMakeMove — αμφίδρομο, κίνηση κομματιού (x,y,toX,toY)
- NetRematch — αίτημα rematch
- NetLobbyRefresh — server shutdown notification
- NetDraw — πρόταση/αποδοχή/απόρριψη ισοπαλίας (value: 1=offer, 0=decline, 2=accept)

3.3.2 LobbyManager

Ορίζει την κλάση HostEntry (plain data object):

- public string name;
- public string ip;
- public ushort port;
- public string pin;
- public DateTime lastSeen;

Χρησιμοποιείται ως value type στο Dictionary<string, HostEntry>

του LobbyDiscovery για αποθήκευση ανακαλυφθέντων hosts. Ένας host θεωρείται "fresh" αν εντοπίστηκε εντός 6 δευτερολέπτων(ώστε να αφαιρείται αν σταματήσει να κάνει broadcast).

3.3.3 Client-Server Flow

Ακολουθία σύνδεσης:

1. HOST: Server.Init(8007, name, pin) → driver.Listen()

LobbyDiscovery.StartBroadcasting()

2. CLIENT: LobbyDiscovery.StartListening() → βρίσκει host

Επιλογή lobby → (PIN dialog αν χρειαστεί)

Client.Init(ip, port) → driver.Connect()

3. Σύνδεση: Client αποστέλλει NetWelcome

4. Server: λαμβάνει NetWelcome → αποστέλλει NetStartGame

(με team assignments: 0=λευκό, 1=μαύρο)

5. Παιχνίδι: NetMakeMove ανταλλάσσεται ανά κίνηση

Rematch flow:

1. Παίκτης πατά REMATCH → NetRematch(teamId) προς server

2. Server: αν και οι 2 πατήσουν → νέο NetStartGame

3.3.4 Synchronization & State Management

Η κατάσταση παιχνιδιού είναι deterministic — αν και οι δύο πλευρές ξεκινήσουν από την ίδια αρχική θέση και εκτελέσουν τις ίδιες κινήσεις με την ίδια σειρά, το αποτέλεσμα είναι πάντα ταυτόσημο. Άρα αρκεί να στέλνεται μόνο η κίνηση (NetMakeMove) και όχι η πλήρης κατάσταση ταμπλό.

Ο MainThreadDispatcher χρησιμοποιείται για ενημέρωση UI από background threads (UDP discovery threads) προς το Unity main thread μέσω ConcurrentQueue<Action>.

4 Υλοποίηση Εφαρμογής

4.1 Παρουσίαση scripts UI

Η εφαρμογή χρησιμοποιεί ένα **δυναμικό σύστημα UI**, όπου κάθε panel δημιουργείται από αντίστοιχη μέθοδο BuildXxxPanel(). Αυτές οι μέθοδοι καλούνται μία φορά στην εκκίνηση της σκηνής (Start()) και τα panels απλά ενεργοποιούνται ή απενεργοποιούνται (SetActive(true/false)) αντί να καταστρέφονται, εξοικονομώντας πόρους και διασφαλίζοντας ομαλές μεταβάσεις.

Το κεντρικό σημείο ελέγχου είναι το singleton GameUI, το οποίο αρχικοποιείται στη Awake() με Instance = this. Στη συνέχεια καλούνται οι builder μέθοδοι για:

- BuildNameInputPanel() – φόρμα για όνομα χρήστη και PIN του Host
- BuildWaitingPanel() – οθόνη αναμονής αντιπάλου
- SetupScrollContent() – λίστα διαθέσιμων lobbies
- BuildLobbyConnectButton() – κουμπί σύνδεσης σε lobby
- BuildConnectionErrorOverlay() – overlay για σφάλματα σύνδεσης
- BuildPinDialog() – εισαγωγή PIN για προστατευμένα δωμάτια

Κάθε panel δημιουργείται ως παιδί του Canvas, με RectTransform που καλύπτει ολόκληρη την οθόνη ανεξάρτητα από ανάλυση. Το Canvas είναι σε

RenderMode.ScreenSpaceOverlay, εξασφαλίζοντας σωστή εμφάνιση σε κινητές συσκευές και PC. Τα στοιχεία των panels οργανώνονται με VerticalLayoutGroup για αυτόματη στοίχιση και σταθερά διαστήματα.

Σημαντική είναι η NameInputPanel, όπου ο χρήστης εισάγει όνομα (έως 20 χαρακτήρες) και PIN (4 ψηφία). Το πεδίο PIN περιορίζεται σε αριθμούς και αφαιρεί τυχόν εμφανιζόμενο μήνυμα σφάλματος μόλις ο χρήστης αρχίσει να πληκτρολογεί.

Η κάμερα χειρίζεται μέσω της ChangeCamera(), που δέχεται τιμές CameraAngle (menu, whiteTeam, blackTeam). Απενεργοποιεί όλες τις κάμερες και ενεργοποιεί μόνο τη σωστή. Στο Local Game η κάμερα παραμένει πάντα στη γωνία whiteTeam.

4.2 Παρουσίαση scripts Chess Logic

Η σκακιστική λογική βρίσκεται στο ChessBord.cs. Η σκακιέρα αναπαρίσταται με πίνακα Piece[8,8], όπου κάθε θέση περιέχει null ή αντικείμενο κομματιού.

Στη Start():

1. Φορτώνονται ήχοι (move.wav, check.wav, castle.wav, promote.wav)
2. Ρυθμίζεται η κάμερα (AdjustCamera()) βάσει aspect ratio
3. Δημιουργείται η σκακιέρα (GenerateBord()) και τοποθετούνται όλα τα κομμάτια (SpawnAllPieces())

Ο χειρισμός εισόδου συνδυάζει touch και mouse μέσω των βοηθητικών IsInputDown(), IsInputUp(), GetInputPosition(), επιτρέποντας cross-platform χρήση χωρίς διπλασιασμό κώδικα.

Ο κύριος βρόχος (Update()) εντοπίζει το τετράγωνο κάτω από τον κέρσορα ή το δάχτυλο με raycast, υπολογίζοντας x και y στο πλέγμα [0,7]. Διακρίνει hover, input down (επιλογή κομματιού) και input up (εκτέλεση ή ακύρωση κίνησης). Κατά το dragging, το κομμάτι ανυψώνεται για οπτική ένδειξη.

Η SimulateMoveForSinglePiece() εξασφαλίζει ότι δεν εκτελείται κίνηση που αφήνει τον Βασιλιά σε σαχ, προσομοιώνοντας κάθε κίνηση σε προσωρινό πίνακα και φιλτράροντας τις μη έγκυρες.

Το τέλος παρτίδας ελέγχεται μετά από κάθε κίνηση:

- Ειδικές κινήσεις (en passant, castling, promotion) εκτελούνται μέσω ProssesSpecialMove()
- CheckForCheckmate() ελέγχει αν ο παίκτης έχει έγκυρες κινήσεις

- Αν υπάρχουν μόνο οι δύο βασιλιάδες → ισοπαλία (DisplayDraw())

Τα αιχμαλωτισμένα κομμάτια τοποθετούνται στις πλατφόρμες deathPlatformWhite/Black μέσω PlaceDeadPiece(), με σωστούς μετασχηματισμούς συντεταγμένων.

Οι ήχοι αναπαράγονται μέσω PlayMoveSound(), με προτεραιότητα: σαχ → ροκέ → προαγωγή → απλή κίνηση.

4.3 Παρουσίαση scripts Networking

Η εφαρμογή χρησιμοποιεί **Unity Transport (UDP με αξιοπιστία)** για να συνδέσει παίκτες σε τοπικό ή online παιχνίδι. Ο **Server** και ο **Client** είναι singleton objects που χειρίζονται τη σύνδεση, την αποστολή και λήψη μηνυμάτων.

Server

- Δημιουργεί τον NetworkDriver, δεσμεύει θύρα και ακούει συνδέσεις.
- Κρατά λίστα με ενεργές συνδέσεις (μέχρι 2 παίκτες).
- Αν υπάρχει όνομα host, ξεκινά broadcast για να γίνει ορατός στο δίκτυο.
- Στην Update():
    1. Στέλνει keep-alive κάθε 20 δευτ.
    2. Ανανεώνει το driver για I/O.
    3. Καθαρίζει αποσυνδεδεμένους clients.
    4. Δέχεται νέες συνδέσεις.
    5. Επεξεργάζεται όλα τα εισερχόμενα μηνύματα.

  Για αποστολή μηνυμάτων: γράφει τα δεδομένα σε DataStreamWriter και στέλνει σε έναν ή πολλούς clients.

Client

- Δημιουργεί NetworkDriver και συνδέεται στον server (IP:port).
- Στην Update() ελέγχει αν η σύνδεση είναι ενεργή και επεξεργάζεται τα εισερχόμενα events.
- Όταν συνδεθεί επιτυχώς, στέλνει μήνυμα **Welcome** για να ενεργοποιηθεί η ανάθεση ομάδων.
- **Μηνύματα**

Υπάρχουν 7 τύποι μηνυμάτων (OpCode):

1. KEEP_ALIVE – έλεγχος σύνδεσης
2. WELCOME – χαιρετισμός client → server
3. START_GAME – εκκίνηση παρτίδας
4. MAKE_MOVE – κίνηση κομματιού
5. REMATCH – αίτημα επανάληψης
6. LOBBY_REFRESH – ανανέωση λίστας lobbies
7. DRAW – πρόταση ή αποδοχή ισοπαλίας

Κάθε μήνυμα κληρονομεί από **NetMessage**, με τρεις μεθόδους:

- Serialize() για αποστολή
- ReceivedOnClient()
- ReceivedOnServer()

  Όταν φτάνει μήνυμα, ενεργοποιεί τα αντίστοιχα **events**, π.χ. το ChessBoard εγγράφεται και ενημερώνεται.

Τοπικό παιχνίδι

Server και Client τρέχουν ταυτόχρονα στην ίδια συσκευή (127.0.0.1), έτσι χρησιμοποιείται ο ίδιος κώδικας για τοπικό και online παιχνίδι.

Lobby Discovery

- Host: στέλνει UDP broadcast κάθε 2 δευτ. και απαντά σε probes client.
- Client: ακούει broadcast και στέλνει probe κάθε ~2 δευτ., λαμβάνει απαντήσεις.
- Τα hosts αποθηκεύονται σε dictionary με χρονικό έλεγχο (6 δευτ.) για να εμφανίζονται μόνο ενεργοί.
- Η ενημέρωση της λίστας στο UI γίνεται στο main thread για ασφάλεια με το Unity API.
- **Ανίχνευση τοπικής IP**
- Προσπαθεί να ανοίξει UDP socket προς 8.8.8.8 για να βρει local IP.
- Αν αποτύχει, σαρώνει τα network interfaces και αγνοεί link-local και loopback διευθύνσεις.

4.4 GameManager και κεντρικός έλεγχος

Δεν υπάρχει ξεχωριστό GameManager script στην εφαρμογή. Αντ' αυτού, οι αρμοδιότητες κατανέμονται ρητά σε τρία components:

- **ChessBord.cs** – κεντρικός έλεγχος παιχνιδιού: κατάσταση σκακιέρας, κινήσεις, ειδικές κινήσεις, ανίχνευση σαχ/ματ, διαχείριση αιχμαλώτων, ηχητικά εφέ, in-game UI.

- **Game UI.cs** – έλεγχος μενού και πλοήγησης: κύριο μενού, online μενού,lobby discovery UI, εκκίνηση local/online παιχνιδιού, αλλαγή κάμερας, εκκίνηση Server/Client.

- **Server.cs / Client.cs** – Singleton components για τη δικτυακή επικοινωνία, προσβάσιμα καθολικά ως Server.Instance / Client.Instance.

Η επικοινωνία μεταξύ των components γίνεται με δύο τρόπους. Ο πρώτος είναιάμεσες κλήσεις μεθόδων (π.χ. server.Init(), client.Init(), GameUI.Instance.ChangeCamera()), κατάλληλες για άμεση, synchronous αλληλεπίδραση. Ο δεύτερο είναι τα static Action events του NetUtility (C_* για client, S_* για server),τα οποία επιτρέπουν χαλαρή σύζευξη: το ChessBord εγγράφεται στα events που το ενδιαφέρουν (π.χ. NetUtility.C_MAKE_MOVE, NetUtility.S_WELCOME) χωρίς να γνωρίζει τίποτα για τη δομή του networking layer.

4.5 Cross-platform build & testing

Android Build:

- Build System: Gradle
- Scripting Backend: IL2CPP (ARM64)
- Minimum API: 26 (Android 8.0)
- Internet Permission: απαιτείται στο AndroidManifest
- (Unity προσθέτει αυτόματα για NetworkDriver)

Windows Build:

- Architecture: x86_64
- Scripting Backend: Mono ή IL2CPP
- Απαιτεί Run as Administrator για θύρες UDP <8000 αν το Windows Firewall είναι ενεργοποιημένο

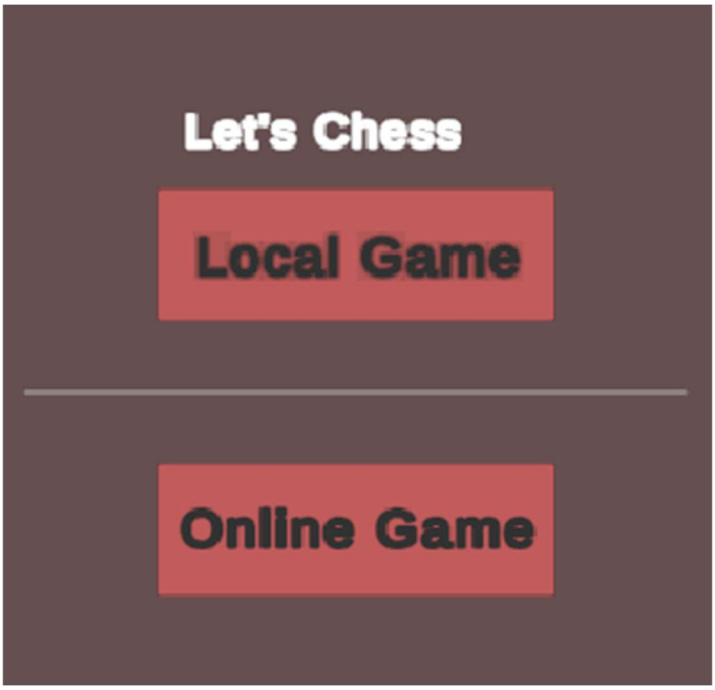
Testing cross-platform:

- Δοκιμές Android→Android (ίδιο Wi-Fi router): πλήρης λειτουργία
- Δοκιμές Android(host)→Windows(client): λειτουργεί μέσω
- ProberLoop + PrePunchGamePort
- Δοκιμές Windows(host)→Android(client): λειτουργεί μέσω
- PassiveListenLoop (Android δέχεται broadcast)

5. Εκτέλεση (Using Manual)

5.1 Αρχική οθόνη

Στην αρχική οθόνη έχουμε 2 επιλογές:



LOCAL GAME :

Εκκίνηση παιχνιδιού για 2 παίκτες στην ίδια συσκευή,

χωρίς σύνδεση δικτύου.

ONLINE GAME:

Μετάβαση στο μενού online παιχνιδιού για σύνδεση

μέσω τοπικού δικτύου.

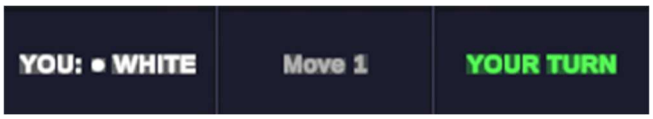
Το κεντρικό μενού συνοδεύεται από ομαλές κινούμενες μεταβάσεις

(animations) κατά την πλοήγηση μεταξύ οθονών.

5.2 Τοπικό Παιχνίδι (Local Game)

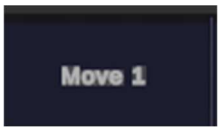
Ξεκινάει κατευθείαν το παιχνίδι και μεταφερόμαστε στην game οθόνη με τη σκακιέρα, καθώς και τις in-game επιλογές:

Άνω μενού:



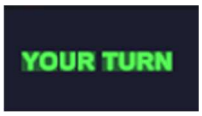
- Χρώμα του παίκτη (YOU: WHITE / YOU: BLACK) (ποιο χρώμα έχει ο παίκτης)

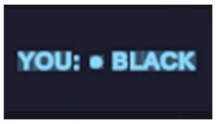
- Αριθμός κίνησης (Move X)

- Κατάσταση:

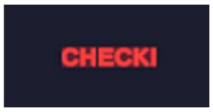
  1. Αν είναι η σειρά σου, θα δεις: YOUR TURN (χρώμα πράσινο)

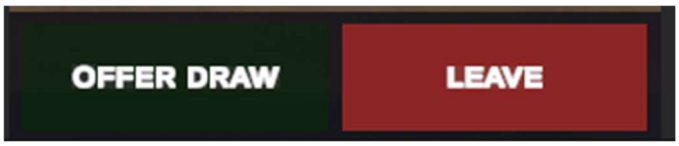
  2. Αν είναι η σειρά του αντιπάλου, θα δεις: 

     Που σημαίνει ότι, επειδή είναι Local Game, είναι η σειρά του άλλου παίκτη.

  3. Σε περίπτωση σαχ: CHECK! (χρώμα κόκκινο)

Κάτω μενού



- OFFER DRAW– Πρόταση ισοπαλίας στον αντίπαλο (απενεργοποιημένο λόγω τοπικού παιχνιδιού)
- LEAVE – Αποχώρηση και επιστροφή στο κύριο μενού

Κεντρική σκακιέρα

- Για να κινήσεις ένα πιόνι, απλώς το σύρεις στη θέση που θέλεις (drag and drop).

- Οι διαθέσιμες κινήσεις εμφανίζονται με πράσινο χρώμα.

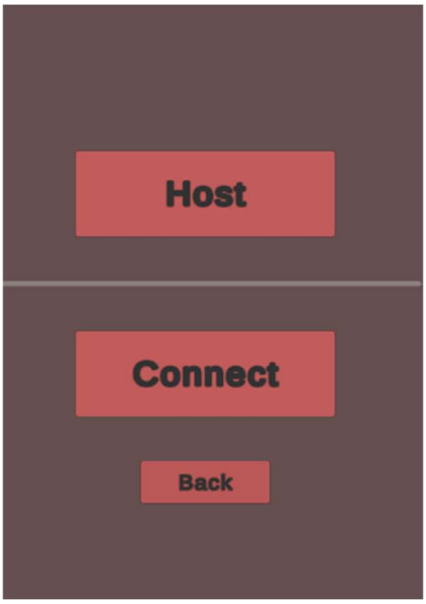
- Αν αφήσεις το πιόνι σε μη επιτρεπτή θέση, επιστρέφει στην αρχική του θέση.

Πλατφόρμες νεκρών πιόνιων

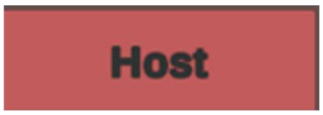
Είναι δύο πλατφόρμες, μία για κάθε παίκτη, στις οποίες τοποθετούνται τα νεκρά πιόνια. Έτσι υπάρχει οπτική επαφή με τα πιόνια που έχεις πάρει και αυτά που σου έχουν πάρει αντίστοιχα.

5.3 Παιχνίδι Μέσω Δικτύου (Online Game)

Υπάρχουν τρεις επιλογές:

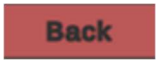


- **Host** – Εμφανίζει το host menu για να προχωρήσεις στη διαδικασία δημιουργίας δωματίου.

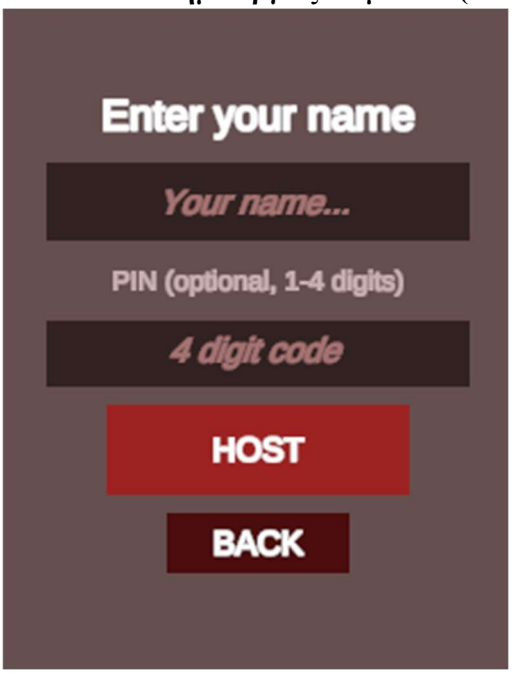


- **Connect** – Σε οδηγεί στο μενού για να δεις και να επιλέξεις διαθέσιμα δωμάτια παιχνιδιού.

- **Back** – Επιστρέφει στο κύριο μενού (Main Menu).



5.3.1 Μενού Δημιουργίας Δωματίου (Host Menu)

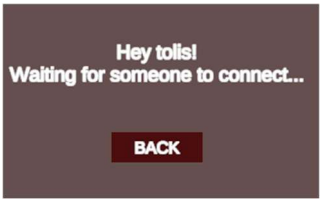


1. **Lobby Name**
   Σε αυτό το πεδίο τοποθετείς το όνομα που θέλεις να εμφανίζεται στους άλλους παίκτες όταν ανοίγουν τη λίστα των lobbies. Μπορεί να έχει έως 20 χαρακτήρες και υποστηρίζει πολλαπλές γλώσσες, ώστε να μπορείς να χρησιμοποιήσεις όποια γλώσσα θέλεις.

2. **Καταχώριση PIN**
   Σε αυτό το πεδίο μπορείς να βάλεις μόνο αριθμούς, έως 4 ψηφία. Χρησιμοποιείται για να κλειδώσεις το δωμάτιο και να μπορούν να παίξουν μόνο όσοι έχουν τον κωδικό. Αν δεν βάλεις PIN, το lobby μένει ανοιχτό και οποιοσδήποτε μπορεί να μπει. Αν επιλέξεις να βάλεις PIN, στο lobby menu θα εμφανίζεται δίπλα στο όνομα του lobby ένα χαρακτηριστικό κίτρινο [PIN].

Κουμπί Host

Πατώντας το κουμπί, το δωμάτιο εμφανίζεται στο δίκτυο και εμφανίζεται μήνυμα αναμονής σύνδεσης κάποιου παίκτη.



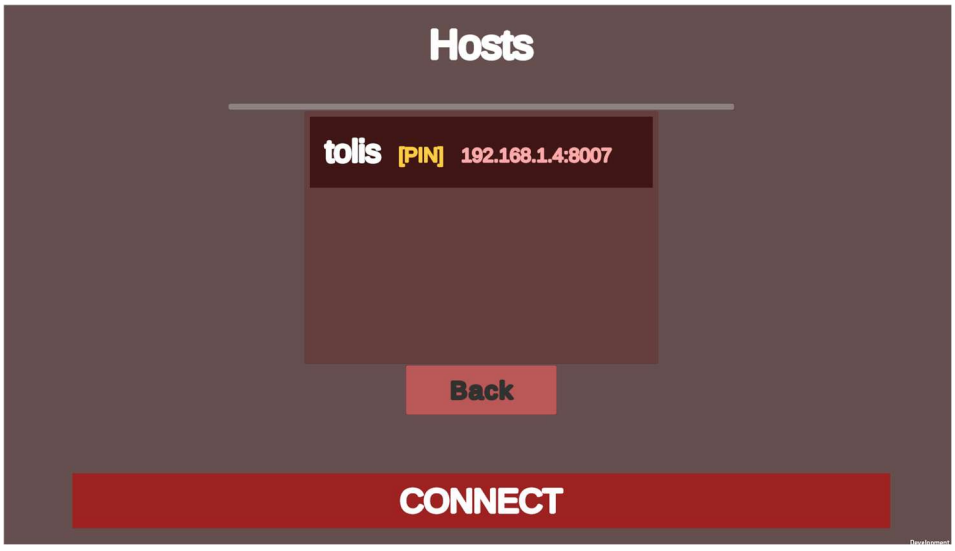
Αν πατήσετε **Back**, επιστρέφετε στην προηγούμενη οθόνη και το lobby διαγράφεται από τη λίστα. Σε περίπτωση ταυτόχρονης σύνδεσης ή τερματισμού του lobby, εμφανίζεται μήνυμα

στον άλλο χρήστη:

Κουμπί Back (host menu)

Επιστρέφει στο Online Menu, δηλαδή στην προηγούμενη σελίδα.

5.3.2 Connect Menu

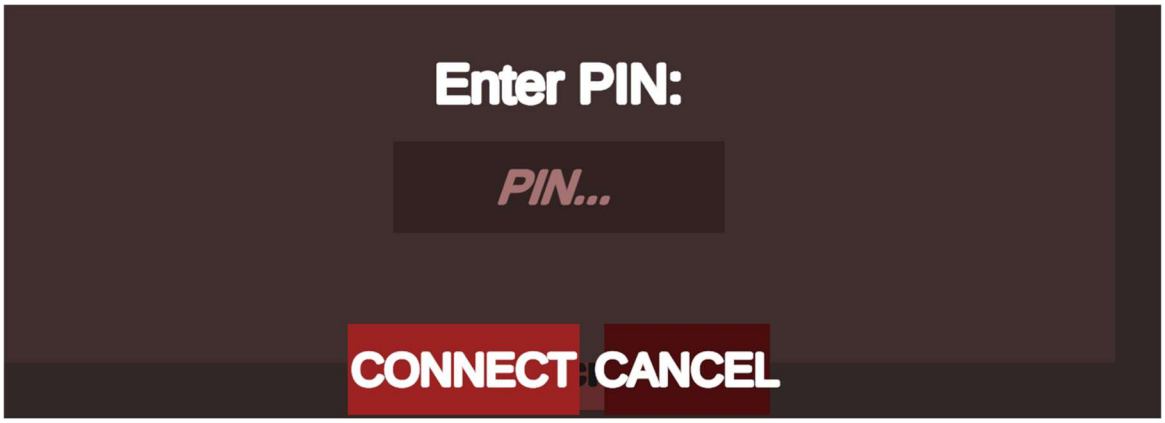


Εδώ βλέπουμε τη λίστα με τα **lobbies** σε ένα scroll view, που επιτρέπει να δεις όλα τα διαθέσιμα δωμάτια παιχνιδιού στο δίκτυο.

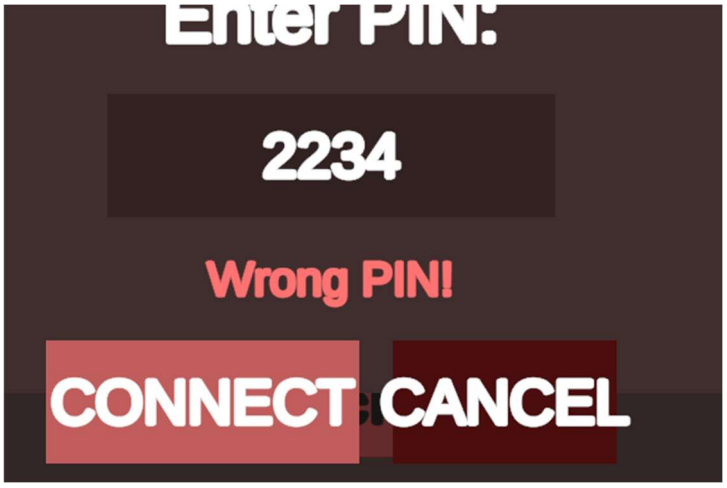
- Το **κίτρινο PIN** αποτελεί αναγνώριστικό ότι το δωμάτιο έχει PIN και δεν είναι ανοικτό σε όλους.

- Πατώντας πάνω σε ένα δωμάτιο, ενεργοποιείται το κουμπί **Connect**.

- Αν το δωμάτιο έχει κωδικό, εμφανίζεται η οθόνη καταχώρισης PIN.



Αν εισάγεις σωστά τον κωδικό, συνδέεσαι και ξεκινάει το παιχνίδι. Σε περίπτωση λάθους, εμφανίζεται μήνυμα σφάλματος.



- Πατώντας **Cancel**, αποεπιλέγεται το lobby και επιστρέφεις στην αρχική οθόνη του Connect Menu.
- Σε περίπτωση ταυτόχρονης σύνδεσης με την ταυτόχρονη κατάργηση του lobby από τον host, εμφανίζεται μήνυμα ότι το δωμάτιο δεν υπάρχει πλέον.

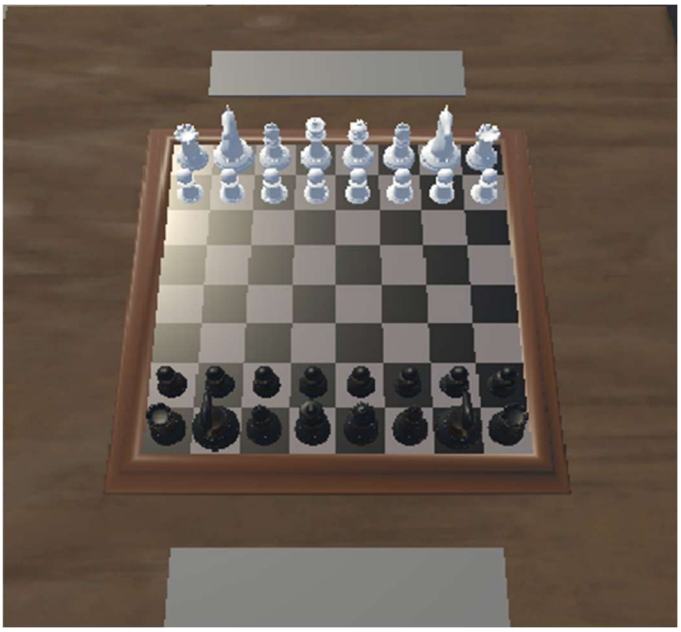
- **5.4 Online In-Game**

Ξεκινάει κατευθείαν το παιχνίδι και μεταφερόμαστε στην game οθόνη με τη σκακιέρα και τις in-game επιλογές:

- **Άνω μενού**

- **Χρώμα του παίκτη** (YOU: WHITE / YOU: BLACK) – ποιο χρώμα έχει ο παίκτης.
    - Ανάλογα με το χρώμα αλλάζει και η οπτική γωνία της σκακιέρας:
        - **Λευκά:** θα εμφανιστεί η σχετική εικόνα.

- **Μαύρα:** θα εμφανιστεί η αντίστοιχη εικόνα.



- **Αριθμός κίνησης** (Move X)

- **Κατάσταση:**

1. Αν είναι η σειρά σου: **YOUR TURN** (χρώμα πράσινο)

2. Αν είναι η σειρά του αντιπάλου: **OPP. TURN** (χρώμα γκρι)

– σημαίνει ότι είναι η σειρά του άλλου παίκτη.

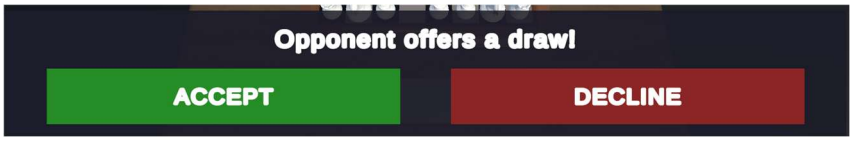
3. Σε περίπτωση σαχ:

CHECK! (χρώμα κόκκινο)

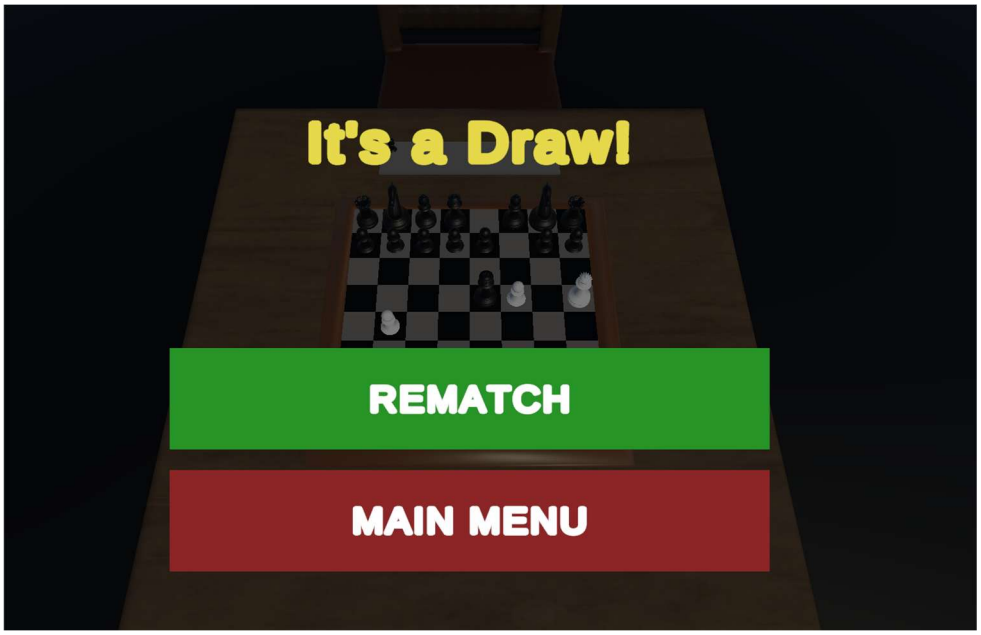
Κάτω μενού

- **OFFER DRAW** – Πρόταση ισοπαλίας στον αντίπαλο.
    - Κάθε παίκτης μπορεί να ζητήσει ισοπαλία έως **3 φορές σε κάθε παιχνίδι**, με διάστημα 15 δευτερολέπτων μεταξύ των προτάσεων.
    - Πατώντας το κουμπί,

ο αντίπαλος βλέπει ειδοποίηση **Accept ή Decline Draw**.



- Αν γίνει **Accept**, το παιχνίδι τελειώνει ως ισοπαλία.

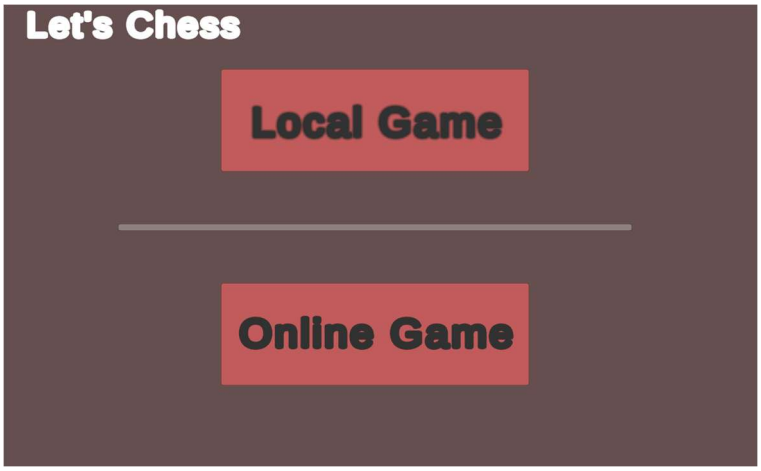


- Αν γίνει **Decline**, το κουμπί **OFFER DRAW** απενεργοποιείται για 15 δευτερόλεπτα και εμφανίζεται μήνυμα ότι η πρόταση απορρίφθηκε.

- Μετά από 3 αποτυχημένες προτάσεις, το κουμπί μπλοκάρεται και δεν μπορείς να προτείνεις ξανά ισοπαλία,

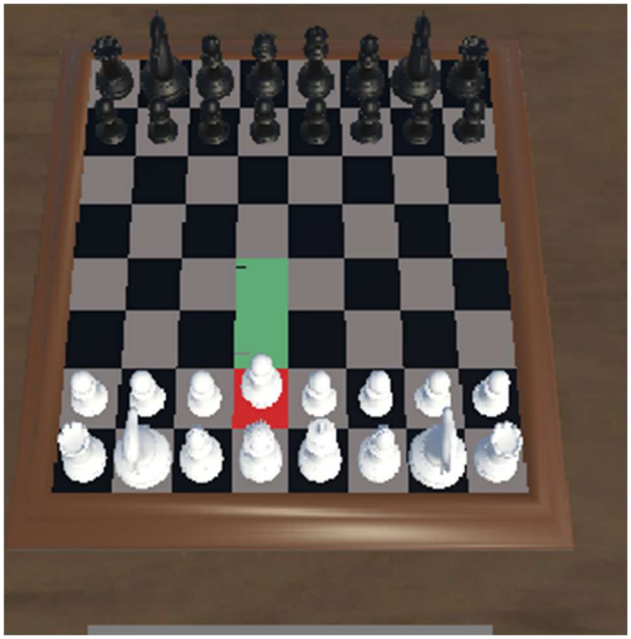
αλλά μπορείς να δεχτείς πρόταση από τον αντίπαλο.

- **LEAVE** – Αποχώρηση και επιστροφή στο κύριο μενού.



Κεντρική σκακιέρα

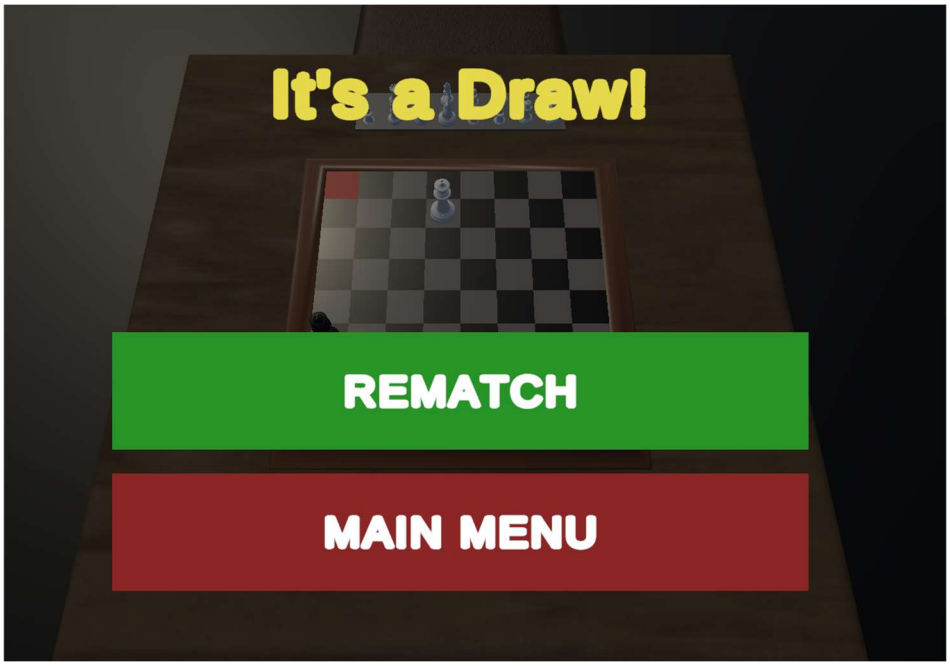
- Για να κινήσεις ένα πιόνι, σύρε το στη θέση που θέλεις (drag and drop).
- Οι διαθέσιμες κινήσεις εμφανίζονται με πράσινο χρώμα.
  Και το επιλεγμένο πιόνι επισημαίνεται,ώντας στον αέρα.



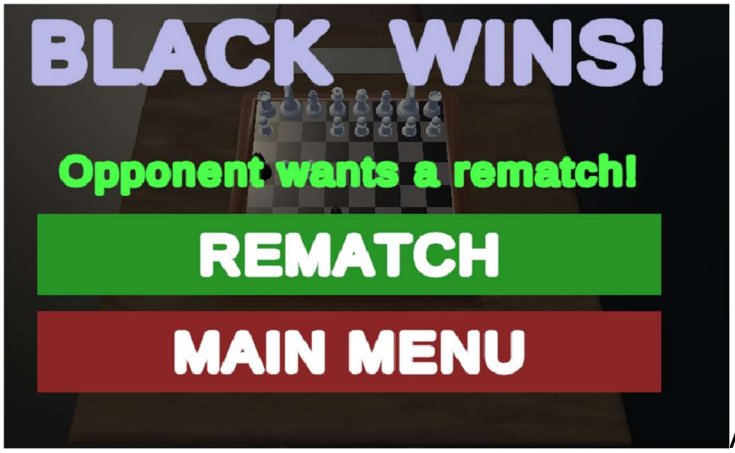
- Αν αφήσεις το πιόνι σε μη επιτρεπτή θέση, επιστρέφει στην αρχική του θέση.

Πλατφόρμες νεκρών πιόνιων

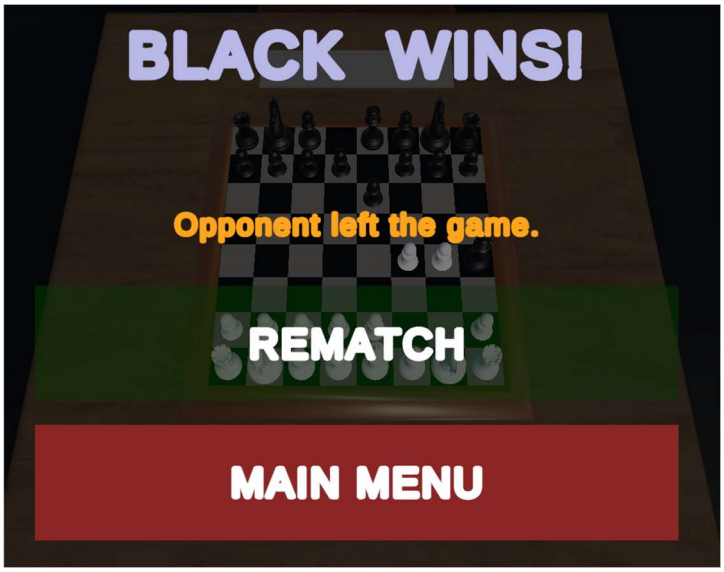
- Υπάρχουν δύο πλατφόρμες, μία για κάθε παίκτη, όπου τοποθετούνται τα νεκρά πιόνια.
- Έτσι υπάρχει οπτική επαφή με τα πιόνια που έχεις πάρει και αυτά που σου έχουν πάρει αντίστοιχα.
- **Τέλος Παιχνιδιού**
    - Αν μείνουν μόνο οι δύο βασιλιάδες, το παιχνίδι τελειώνει ισοπαλία.



REMATCH – Ο αντίπαλος ενημερώνεται ότι θέλεις να παίξετε ξανά.

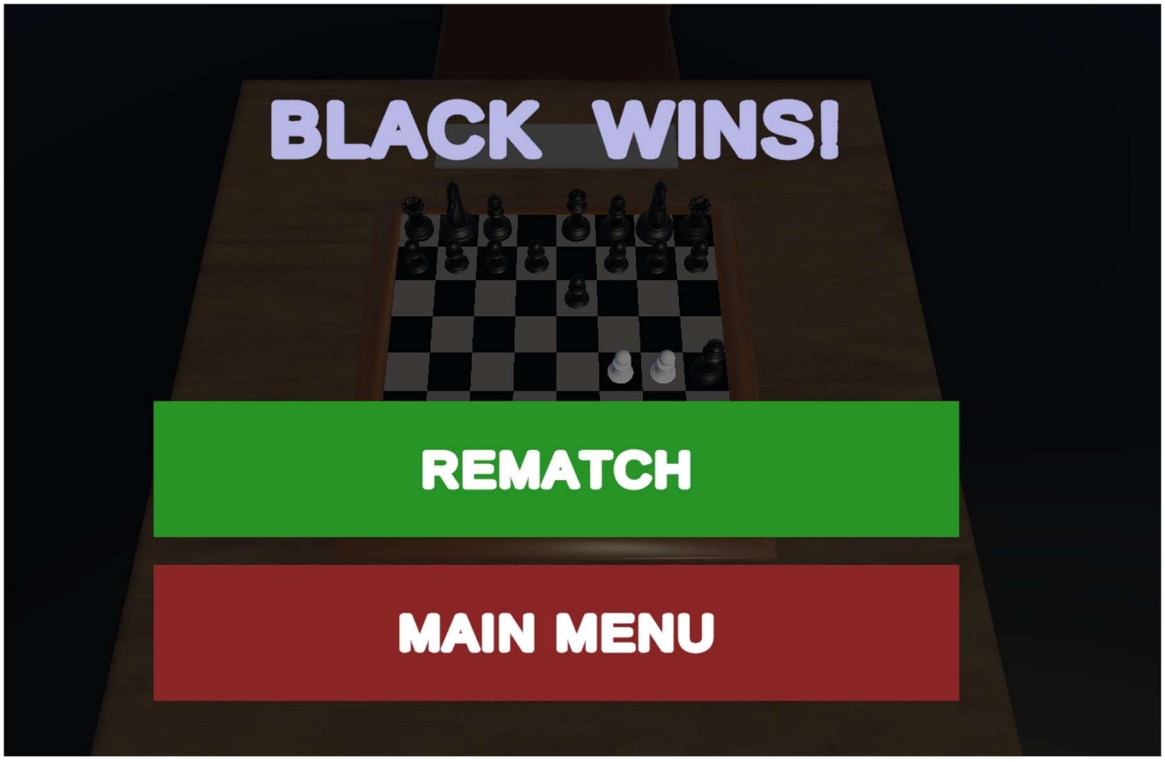


Αν δεχθεί, ξεκινάει νέο παιχνίδι. (Τυχαίο ποιος θα έχει τα μάυρα και ποίος θα έχει τα άσπρα,όπως και στην αρχή)
Αν όχι, εμφανίζεται σχετικό μήνυμα.

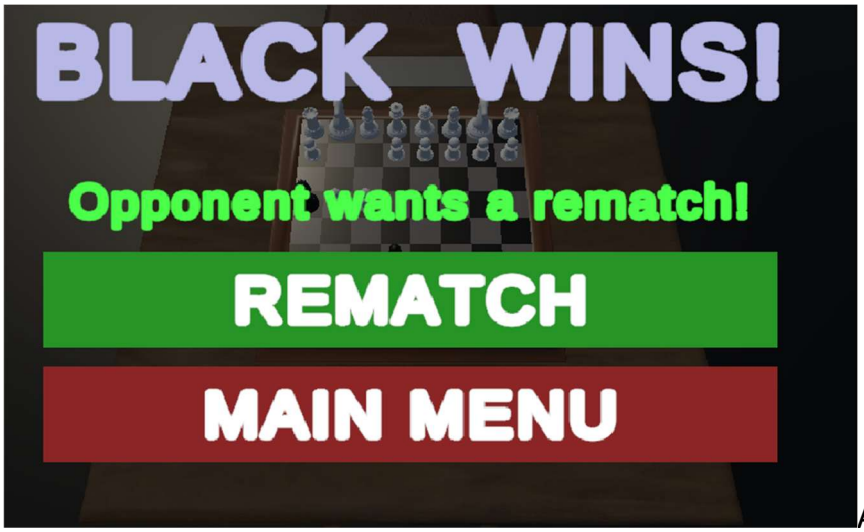


- **LEAVE** – Επιστροφή στο κύριο μενού.
-

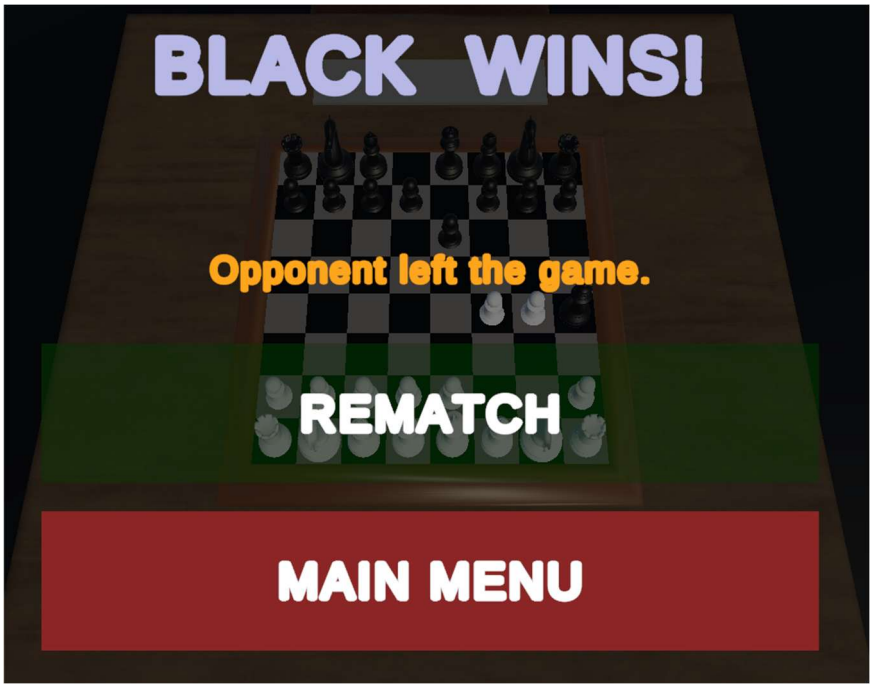
- Σε περίπτωση σαχ-ματ, εμφανίζεται η οθόνη **Νίκης ή Ήττας**, με επιλογές:



- **REMATCH** – Ο αντίπαλος ενημερώνεται ότι θέλεις να παίξετε ξανά.



Αν δεχθεί, ξεκινάει νέο παιχνίδι. (Τυχαίο ποιος θα έχει τα μάυρα και ποίος θα έχει τα άσπρα,όπως και στην αρχή)
Αν όχι, εμφανίζεται σχετικό μήνυμα.



- **LEAVE** – Επιστροφή στο κύριο μενού.

Σημείωση: Κατά τη διάρκεια του παιχνιδιού, το lobby δεν μπορεί να εμφανιστεί σε άλλον χρήστη και διαγράφεται μετά το πέρας του παιχνιδιού, ανάλογα με την απόφαση για rematch.

5.5 Ειδικές Κινήσεις

1. **(Castling – Ροκέ)**

Το **ροκέ** είναι μια ειδική κίνηση στο σκάκι που περιλαμβάνει τον βασιλιά και έναν πύργο, με τους εξής κανόνες:

- Ο βασιλιά και ο πύργος που θα χρησιμοποιηθεί δεν πρέπει να έχουν κινηθεί νωρίτερα στο παιχνίδι.
- Δεν πρέπει να υπάρχει κανένα πιόνι ή άλλο κομμάτι ανάμεσα στον βασιλιά και τον πύργο.
- Ο βασιλιάς δεν πρέπει να βρίσκεται σε σαχ, ούτε να περνάει πάνω από τετράγωνα που απειλούνται από πιόνια του αντιπάλου.
- Η κίνηση γίνεται ως εξής:

- Ο βασιλιάς μετακινείται δύο τετράγωνα προς τον πύργο.

- Ο πύργος μεταφέρεται αμέσως δίπλα στον βασιλιά, στο τετράγωνο που «πέρασε» ο βασιλιάς.

- Το ροκέ μπορεί να γίνει είτε **μακρύ (queenside)** είτε **βραχύ (kingside)**, ανάλογα με τον πύργο που χρησιμοποιείται.

2. Promotion (Προαγωγή)

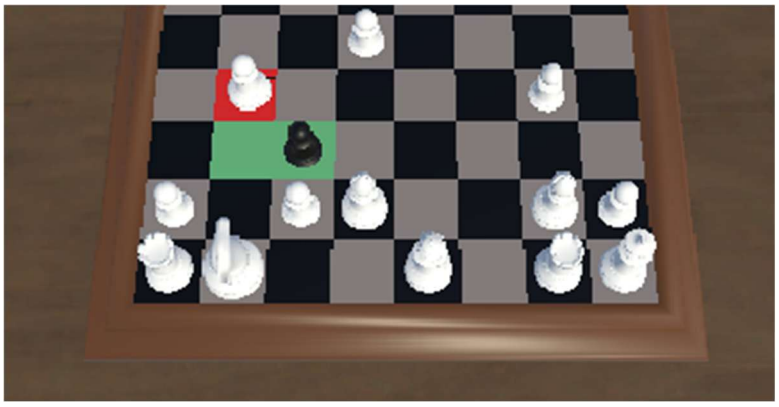
Στην εφαρμογή **LETSCHESS**, όταν ένα πιόνι φτάσει στην τελευταία γραμμή του αντιπάλου, **η προαγωγή γίνεται πάντα σε βασίλισσα**. Οι κανόνες είναι:

- Το πιόνι αντικαθίσταται **αυτόματα από μια βασίλισσα** χωρίς επιλογή από τον παίκτη.
- Η νέα βασίλισσα αποκτά **όλες τις κινήσεις και δυνατότητες της βασίλισσας** αμέσως.
- Η προαγωγή γίνεται **μόλις το πιόνι φτάσει στην αντίπαλη τελευταία γραμμή**, και η κίνηση ολοκληρώνεται με την εμφάνιση της βασίλισσας στη θέση του πιόνιου.

3. En Passant (Εν Πασάν)

Το **en passant** είναι μια ειδική κίνηση που αφορά τα **πεζά** και επιτρέπει σε ένα πιόνι να «πιάνει στο πέρασμα» ένα αντίπαλο πιόνι υπό συγκεκριμένες συνθήκες:

1. Ισχύει μόνο όταν **ένα αντίπαλο πιόνι μετακινείται δύο τετράγωνα μπροστά από τη θέση εκκίνησής του**, περνώντας δίπλα από το δικό σου πιόνι.
2. Το δικό σου πιόνι μπορεί να το **πιάσει σαν να είχε κινηθεί μόνο ένα τετράγωνο** προς τα εμπρός.
3. Η κίνηση πρέπει να γίνει **αμέσως στην επόμενη κίνηση**. Αν δεν γίνει τότε, χάνεται η δυνατότητα για en passant.
4. Το πιόνι που «πιάστηκε» αφαιρείται από τη σκακιέρα και το δικό σου πιόνι καταλαμβάνει το τετράγωνο που πέρασε το αντίπαλο πιόνι.



6. Συμπεράσματα

6.1 Σύγκριση στόχων με τελικό αποτέλεσμα

Αρχικοί στόχοι vs Αποτέλεσμα:

✓ **Πλήρης σκακιστική λογική:** Υλοποιούνται όλες οι κινήσεις σύμφωνα με τους κανόνες της FIDE, συμπεριλαμβανομένων

castling, en passant, προαγωγής πεζού, σαχ, σαχ-ματ και αυτόματης ισοπαλίας. Δοκιμάστηκε σε όλα τα σενάρια και λειτουργεί σωστά.

✓ **Real-time online παιχνίδι μέσω LAN:** Οι παίκτες μπορούν να συνδεθούν στο ίδιο τοπικό δίκτυο και να παίξουν άμεσα, με αξιόπιστο συγχρονισμό κινήσεων.

✓ **Cross-platform από ενιαίο codebase:** Ο ίδιος κώδικας παράγει εκτελέσιμο για Windows και Android χωρίς αλλαγές στη λογική παιχνιδιού, χάρη στην αφαίρεση εισόδου (touch/mouse) και τη χρήση του Unity Transport.

✓ **Αυτόματη ανακάλυψη lobby:** Οι παίκτες δεν χρειάζεται να γνωρίζουν IP – η εφαρμογή βρίσκει αυτόματα διαθέσιμους hosts στο δίκτυο.

✓ **Σύστημα ισοπαλίας:** Κάθε παίκτης μπορεί να προτείνει ισοπαλία. Ο αντίπαλος μπορεί να την αποδεχτεί ή να την απορρίψει. Αν δεν υπάρχει δυνατότητα σαχ-ματ, η ισοπαλία κηρύσσεται αυτόματα.

✓ **Προστασία lobby με PIN:** Ο host μπορεί να ορίσει τετραψήφιο κωδικό ώστε μόνο όσοι τον γνωρίζουν να μπορούν να συνδεθούν.

✓ **Ηχητικά εφέ:** Αναπαράγονται ήχοι για βασικές ενέργειες (κίνηση κομματιού, σαχ, castling, προαγωγή), προσθέτοντας ζωντάνια στο παιχνίδι.

~**Προαγωγή πεζού — απλοποιημένη:** Τα πεζά προάγονται αυτόματα σε Βασίλισσα χωρίς επιλογή άλλου κομματιού, που είναι στις περισσότερες περιπτώσεις η καλύτερη επιλογή.

6.2 Απόκτηση γνώσεων

Πέραν του τελικού αποτελέσματος, η ανάπτυξη της εφαρμογής αποτέλεσε μια ιδιαίτερα εμπλουτιστική εμπειρία από εκπαιδευτική σκοπιά. Κατά τη διάρκεια της εργασίας αποκτήθηκαν γνώσεις σε πολλαπλούς τομείς.
Σχετικά με το Unity Engine αποκτήθηκε βαθιά κατανόηση του κύκλου ζωής των MonoBehaviour components (Awake, Start, Update), του συστήματος Camera και RayCasting σε τρισδιάστατο χώρο, της διαχείρισης 3D transforms και materials, καθώς και του Canvas/UI συστήματος .

Στον τομέα του προγραμματισμού σε C# η εργασία απαίτησε την ενασχόληση με προχωρημένα θέματα όπως ο πολυνηματικός προγραμματισμός (threading) για το σύστημα ανακάλυψης lobby, η χρήση delegates και events για την επικοινωνία μεταξύ components, καθώς και ο αντικειμενοστραφής σχεδιασμός με κληρονομικότητα για την ιεραρχία των κομματιών.

Ιδιαίτερα σημαντικές αποδείχθηκαν οι γνώσεις που αποκτήθηκαν στον τομέα των δικτύων. Η κατανόηση της διαφοράς μεταξύ UDP broadcast και unicast, η αντιμετώπιση ζητημάτων firewall σε Windows, και η εξοικείωση με το Unity Transport API αποτέλεσαν ουσιαστικές προκλήσεις που αντιμετωπίστηκαν αποτελεσματικά μέσω μελέτης και πειραματισμού.

Τέλος, αξιόλογη ήταν η εμπειρία που αποκτήθηκε στην ανάπτυξη εφαρμογών για πολλαπλές πλατφόρμες: η διαχείριση διαφορετικών ρυθμίσεων build, η αφαίρεση της εισόδου χρήστη ώστε να λειτουργεί ομοιόμορφα σε αφή και ποντίκι.

- Unity Engine: MonoBehaviour lifecycle, Camera Ray Casting,
- 3D Transforms, Materials, Canvas/UI system
- C# Advanced: Threading, async patterns, delegates/events,

- generics, ConcurrentQueue
- Networking: UDP sockets, broadcast/unicast, Windows Firewall
- bypass techniques, Unity Transport API
- OOP Design: Inheritance (Piece hierarchy), Singleton pattern,
- Observer pattern (events), Separation of Concerns
- Cross-platform development: Platform-specific build settings,
- input abstraction, network differences Android/Windows

6.3 Περιορισμοί εφαρμογής

Παρά την επιτυχή υλοποίηση των βασικών στόχων, η εφαρμογή παρουσιάζει ορισμένους περιορισμούς που αξίζει να αναφερθούν με ειλικρίνεια.

Ο πιο σημαντικός περιορισμός αφορά το εύρος της δικτύωσης: η εφαρμογή λειτουργεί αποκλειστικά εντός τοπικού δικτύου (LAN/Wi-Fi) .
Εξίσου αξιοσημείωτη είναι η απουσία τεχνητής νοημοσύνης. Ο χρήστης δεν μπορεί να παίξει μόνος του εναντίον της εφαρμογής, καθώς δεν υπάρχει υλοποιημένος αλγόριθμος AI. Αυτό σημαίνει ότι για να ξεκινήσει παρτίδα απαιτείται πάντα η παρουσία δεύτερου ανθρώπινου παίκτη.

Η εφαρμογή επίσης δεν αποθηκεύει το ιστορικό παρτίδων, δεν διαθέτει σύστημα λογαριασμών ή κατάταξης παικτών, και δεν υποστηρίζει την πλατφόρμα iOS.
Τέλος, η προαγωγή πεζού δεν δίνει επιλογή κομματιού στον παίκτη, περιορισμός που αναφέρθηκε ήδη στην προηγούμενη ενότητα.

1. Μόνο 2 παίκτες (δεν υποστηρίζεται spectator mode)
2. Μόνο LAN — δεν λειτουργεί μέσω internet
3. Δεν υπάρχει AI αντίπαλος (computer player)
4. Δεν υποστηρίζεται επιλογή κομματιού κατά Προαγωγή

5. Δεν αποθηκεύεται ιστορικό παρτίδων (PGN export)
6. Δεν υπάρχει σύστημα ELO / ranking
7. Δεν υποστηρίζεται iOS

6.4 Προτάσεις μελλοντικής εξέλιξης

Η παρούσα εφαρμογή αποτελεί μια ολοκληρωμένη βάση πάνω στην οποία μπορούν να προστεθούν σημαντικές επεκτάσεις στο μέλλον. Οι πιο ουσιαστικές προτάσεις παρουσιάζονται παρακάτω.

Η υλοποίηση AI αντιπάλου θα ήταν η πιο σημαντική βελτίωση, δίνοντας στον χρήστη τη δυνατότητα να εξασκείται και χωρίς δεύτερο παίκτη. Ένας αλγόριθμος minimax με alpha-beta pruning αποτελεί τη βασική προσέγγιση για σκακιστικό AI και θα μπορούσε να ενσωματωθεί στο υπάρχον σύστημα κινήσεων.

Η επέκταση σε internet multiplayer θα αντιμετώπιζε τον πιο ουσιαστικό περιορισμό της εφαρμογής. Τεχνολογίες όπως το Photon, το Mirror Networking ή ένας dedicated relay server θα επέτρεπαν σε παίκτες από διαφορετικές τοποθεσίες να συνδέονται μέσω διαδικτύου.

Η προσθήκη χρονομέτρου ανά παίκτη (chess clock) θα προσέδιδε περισσότερη αγωνιστική διάσταση στο παιχνίδι, καθιστώντας το κατάλληλο για τυπικές σκακιστικές αντιπαραθέσεις με χρονικό όριο.

Επιπλέον, ένα σύστημα αποθήκευσης παρτίδων σε μορφή PGN θα επέτρεπε στους χρήστες να αποθηκεύουν και να αναθεωρούν τις παρτίδες τους, ενώ η επιλογή κομματιού κατά την προαγωγή θα συμπλήρωνε την πλήρη υλοποίηση των κανόνων.

Τέλος, η υποστήριξη του iOS ως τρίτης πλατφόρμας, η προσθήκη εναλλακτικών θεμάτων σκακιέρας και η δυνατότητα spectator mode για παρατηρητές χωρίς δικαίωμα κίνησης θα αποτελούσαν ουσιαστικές βελτιώσεις εμπειρίας χρήστη.

1. AI αντίπαλος (minimax algorithm με alpha-beta pruning)
2. Internet multiplayer μέσω dedicated relay server ή WebRTC (Normcore, Mirror Networking, Photon)
3. Σύστημα λογαριασμών & ELO ranking
4. Χρονόμετρο ανά παίκτη (chess clock)
5. Επιλογή κομματιού κατά Προαγωγή
6. PGN export (αποθήκευση/φόρτωση παρτίδων)
7. iOS build support
8. Animations κομματιών και ηχητικά εφέ
9. Θέματα σκακιέρας (εναλλακτικά χρώματα/υφές)
10. Spectator mode (παρατηρητής συνδέεται read-only)

7. Βιβλιογραφία & Παραρτήματα

7.1 Βιβλιογραφία

I. FIDE. "Laws of Chess." World Chess Federation, 2023.
https://www.fide.com/FIDE/handbook/LawsOfChess.pdf

II. Unity Technologies. "Unity Manual — Unity Transport Package."
https://docs.unity3d.com/Packages/com.unity.transport@latest

III. TextMeshPro Documentation. Unity Technologies
https://docs.unity3d.com/Packages/com.unity.textmeshpro@latest

IV. Netcode for GameObjects

https://docs.unity3d.com/6000.3/Documentation/Manual/com.unity.netcode.gameobjects.html

7.2 video

1. https://www.youtube.com/watch?v=qVhG6ZWqD-o&list=PLmcbjnHce7SeAUFouc3X9zqXxiPbCz8Zp
2. https://www.youtube.com/watch?v=kVt0I6zZsf0&t=60s
3. https://www.youtube.com/watch?v=lFZeeTZ29w0&list=PLXV-vjyZiT4b7WGjgiqMy422AVyMaigl1
4. https://www.youtube.com/watch?v=B8eNP15EeYY

7.3 Scripts

Λογική:

- Assets/Scripts/ChessBord.cs
- Assets/Scripts/ChessObjects/Piece.cs
- Assets/Scripts/ChessObjects/SubPiece/Pawn.cs
- Assets/Scripts/ChessObjects/SubPiece/Rook.cs
- Assets/Scripts/ChessObjects/SubPiece/Knight.cs
- Assets/Scripts/ChessObjects/SubPiece/Bishop.cs
- Assets/Scripts/ChessObjects/SubPiece/Queen.cs
- Assets/Scripts/ChessObjects/SubPiece/King.cs

Δικτύωση:

- Assets/Scripts/Network/Server.cs
- Assets/Scripts/Network/Client.cs
- Assets/Scripts/Network/LobbyDiscovery.cs
- Assets/Scripts/Network/LobbyManager.cs
- Assets/Scripts/Network/NetUtility.cs
- Assets/Scripts/Network/MainThreadDispatcher.cs

Μηνύματα Δικτύου:

- Assets/Scripts/Network/Messages/NetMessages.cs
- Assets/Scripts/Network/Messages/NetKeepAlive.cs
- Assets/Scripts/Network/Messages/NetWelcome.cs
- Assets/Scripts/Network/Messages/NetStartGame.cs

- Assets/Scripts/Network/Messages/NetMakeMove.cs
- Assets/Scripts/Network/Messages/NetRematch.cs
- Assets/Scripts/Network/Messages/NetLobbyRefresh.cs
- Assets/Scripts/Network/Messages/NetDraw.cs

Διεπαφή Χρηστών:

- Assets/Scripts/Ui/Game UI.cs

8. Περαιτέρω σύνδεσμοι και αρχεία

GitHub Project Link +(.apk +.exe)

https://github.com/ApostolisSipsas/LETSCHESS_FILES

9. Ευχαριστίες

Θα ήθελα να εκφράσω την ειλικρινή μου ευγνωμοσύνη στον **κ. Ευθύμιο Απέλη**, επιβλέποντα καθηγητή της παρούσας πτυχιακής εργασίας, για την καθοδήγηση, την πολύτιμη καθοδήγηση και τις εποικοδομητικές συμβουλές καθ' όλη τη διάρκεια της εκπόνησης της εργασίας. Η επιστημονική του γνώση, η υπομονή και η συνεχής ενθάρρυνση υπήρξαν καθοριστικές για την επιτυχή ολοκλήρωση του έργου και τη βελτίωση τόσο της τεχνικής όσο και της συγγραφικής μου ικανότητας.

Επιπλέον, θα ήθελα να ευχαριστήσω το **Τμήμα Πληροφορικής του Πανεπιστημίου Πειραιά** που κατέστησε δυνατή την ανάπτυξη και τις δοκιμές της εφαρμογής. Η γνώση και η εμπειρία που αποκόμισα κατά τη διάρκεια των σπουδών μου στο Πανεπιστήμιο αποτέλεσαν ουσιαστική βάση για την εκπόνηση της παρούσας εργασίας.

Τέλος, ευχαριστώ την οικογένεια και τους φίλους μου για την υποστήριξη, την κατανόηση και την ενθάρρυνσή τους, οι οποίοι συνέβαλαν σημαντικά στη διατήρηση της επιμονής και του ενδιαφέροντος μου καθ’ όλη τη διάρκεια αυτής της προσπάθειας.